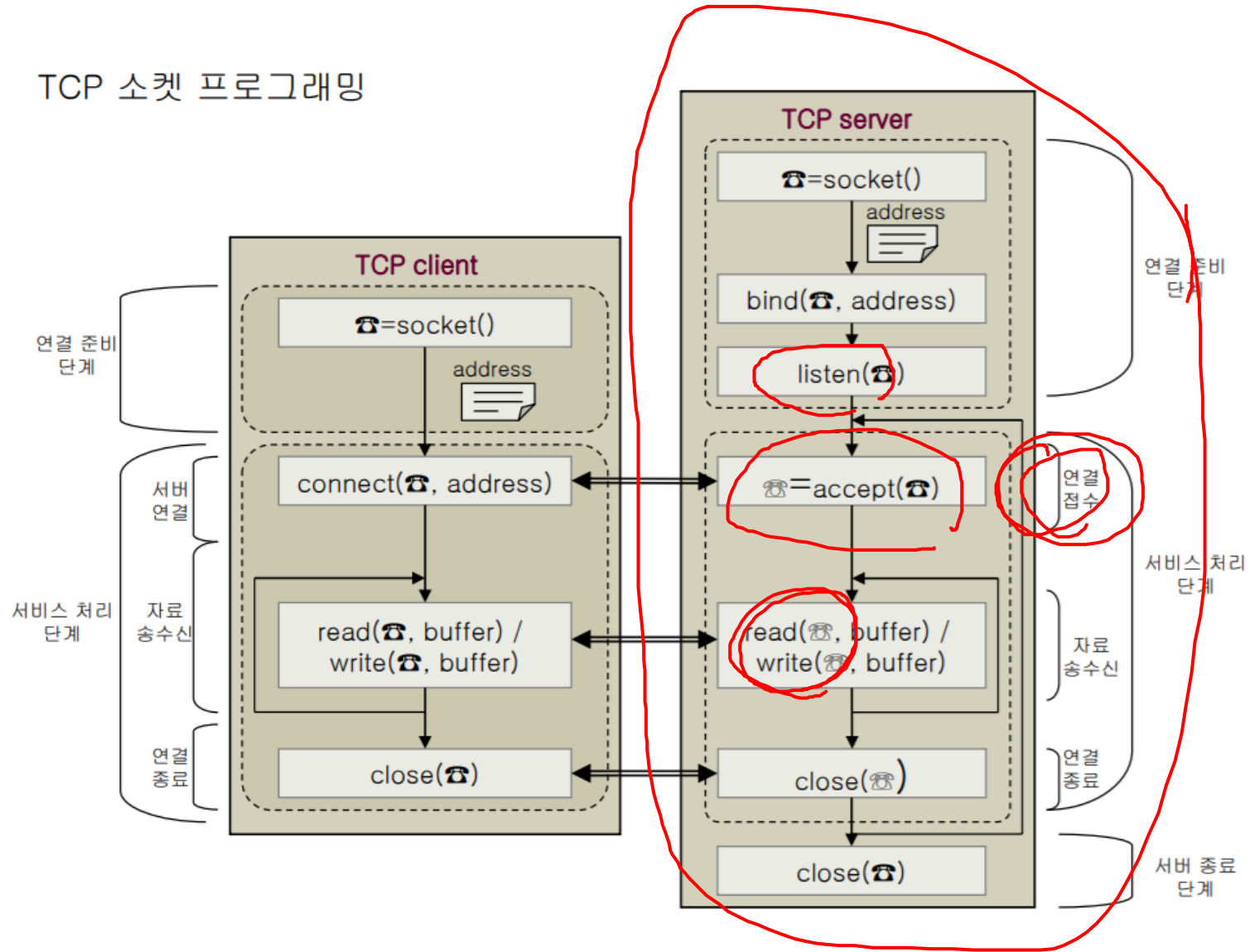


소켓 프로그래밍

6장 UDP 소켓 프로그래밍

소켓 프로그래밍

TCP 소켓 프로그래밍



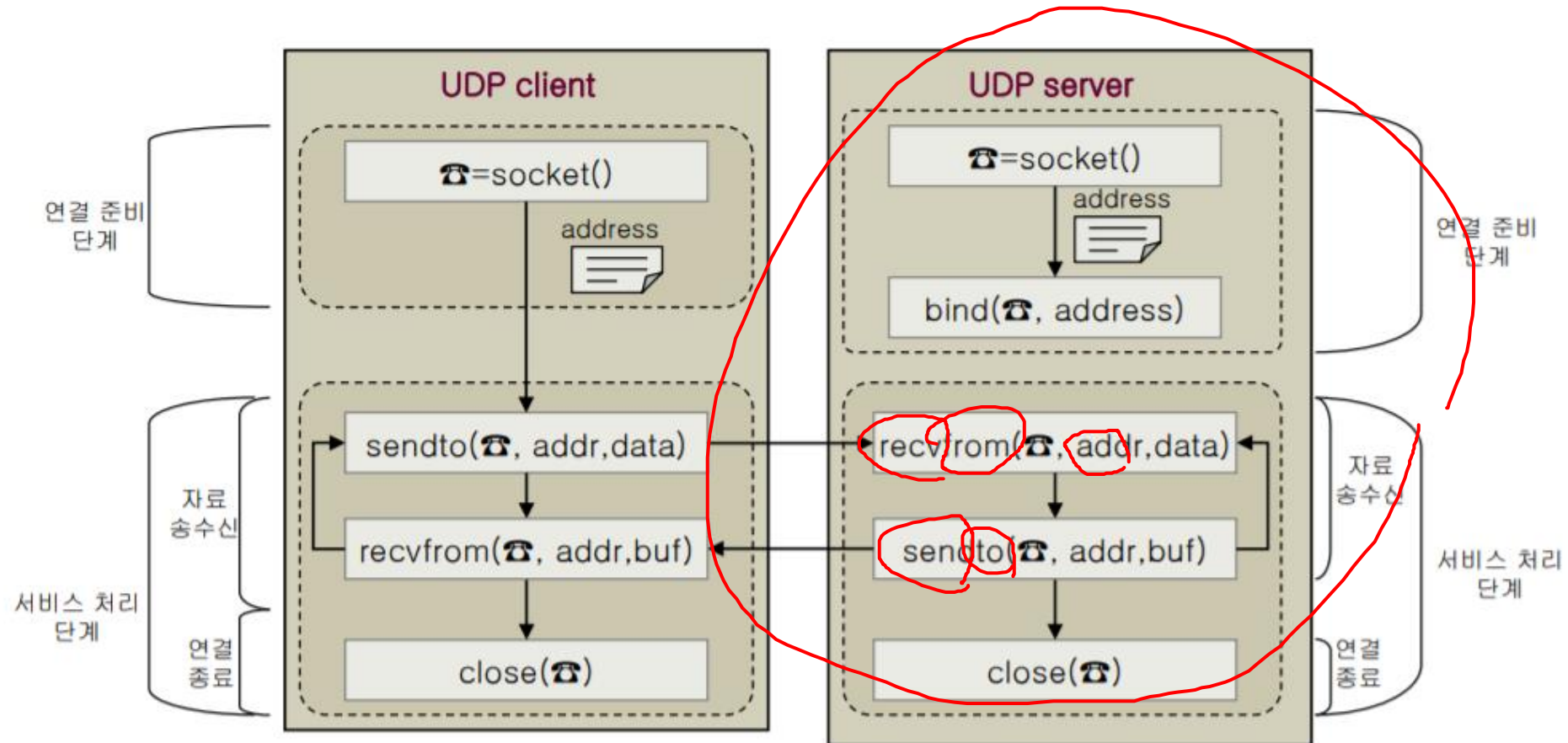
소켓 프로그래밍

TCP v.s. UDP

	TCP	UDP
신뢰성 reliability	● 오류제어 (사본 보유 및 재전송) ● 흐름제어 (flow control)	● 오류제어 기능 없음 ● 흐름제어 기능 없음
	신뢰성이 높다 (reliable)	신뢰성이 낮다 (unreliable)
연결 connection	● 경로 설정 (path setup) ● 자료의 순서 유지	● 경로 설정 절차가 없음 ● 자료의 순서가 뒤바뀔 수 있음
	연결형(connection-oriented)	비연결형(connection-less)
비용 cost	● more space, more time	● less space, less time
	고비용	저비용
응용 분야 application		● 한 패킷으로 서비스(DNS, time) ● 안전한 LAN 환경 (NFS)
	TCP	UDP

소켓 프로그래밍

UDP 소켓 프로그래밍

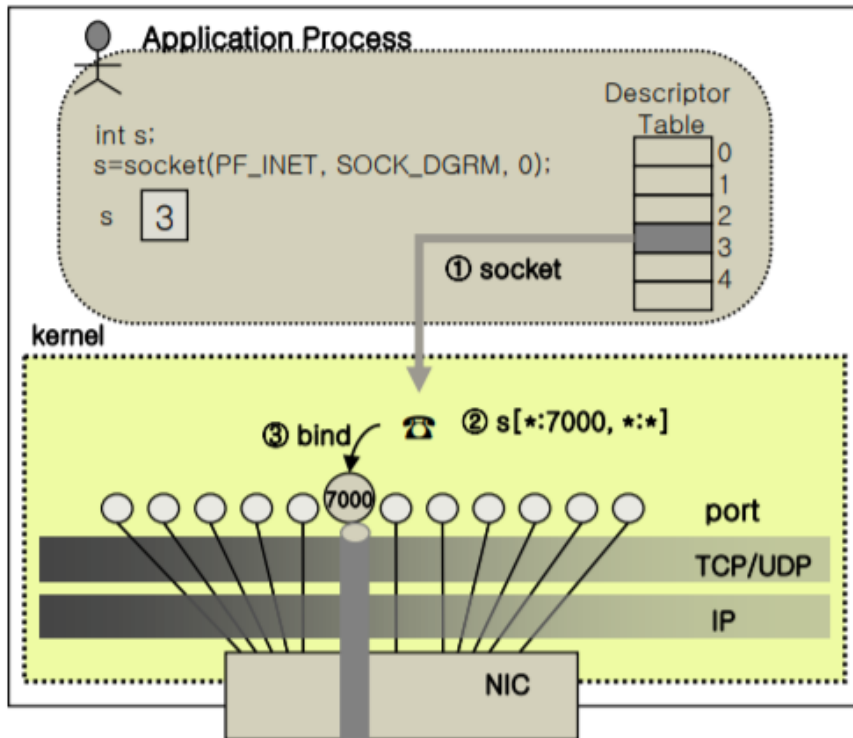


소켓 프로그래밍

UDP 서버 프로그램의 기능 수행

[연결 준비 단계]

- ① 소켓 생성(socket) : 네트워크 통신을 위해 소켓을 생성한다.
- ② 연결할 주소 준비 : 클라이언트로부터 서비스 요청을 접수할 서버의 주소(IP 주소와 포트 번호)를 결정하여 구조체 변수에 저장한다.
- ③ 주소 연결(bind) : 소켓에 프로토콜 주소를 할당하여 실제로 연결한다.



```
int s;  
struct sockaddr_in s_addr;
```

① UDP 소켓 생성(socket)

```
s = socket(PF_INET, SOCK_DGRAM, 0);
```

② 연결할 주소 준비

```
memset(&s_addr, 0, sizeof(s_addr));  
s_addr.sin_addr.s_addr = inet_addr(INADDR_ANY);  
s_addr.sin_family = AF_INET;  
s_addr.sin_port = htons(7000);
```

③ 주소에 연결(bind)

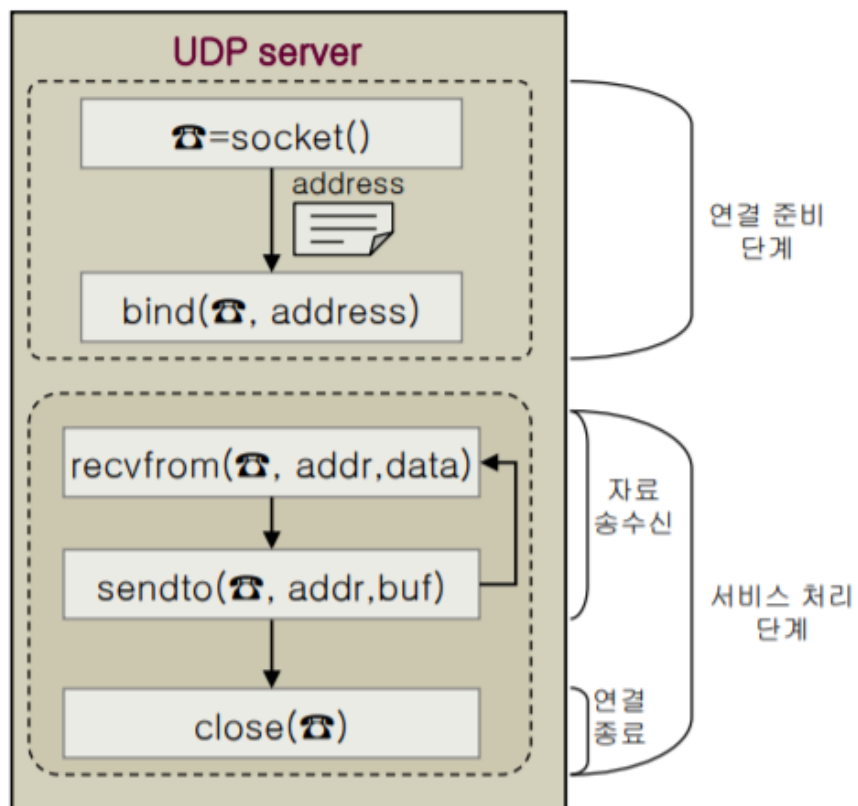
```
bind(s, (struct sockaddr *)&s_addr, sizeof(s_addr));
```

소켓 프로그래밍

UDP 서버 프로그램의 기능 수행

[서비스 처리 단계]

- ① 자료의 송수신(recvfrom/sendto) : UDP 클라이언트와 자료 송수신
- ② 연결 종료(close/shutdown) : 클라이언트와의 연결 소켓 종료



① UDP 자료의 송수신

UDP는 메시지 경계를 보존

- 한 번 호출한 `recvfrom()`은 한 번 `sendto()`한 자료 반환 / `sendto()`와 `recvfrom()` 호출 횟수가 일치

- 여러 클라이언트로 부터 동시에 자료가 도착 / 발신자가 구분되도록 메시지 경계 유지

커널 버퍼에 복사하지 않음

② 연결 종료 단계

`close (c) ;`

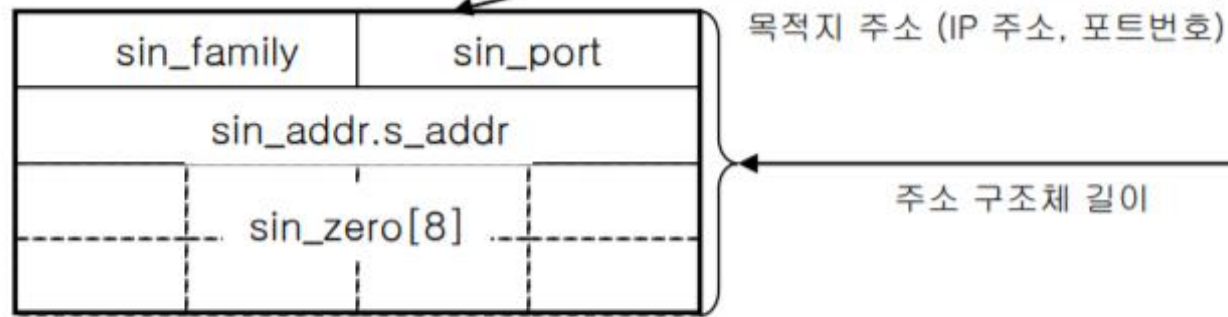
소켓 프로그래밍

UDP 서버 프로그램의 기능 수행

[서비스 처리 단계]

① 자료의 송신(sendto)

```
int sendto(int socket, const void *buf, int nbyte, unsigned int flags, struct sockaddr *to, int addrlen);
```



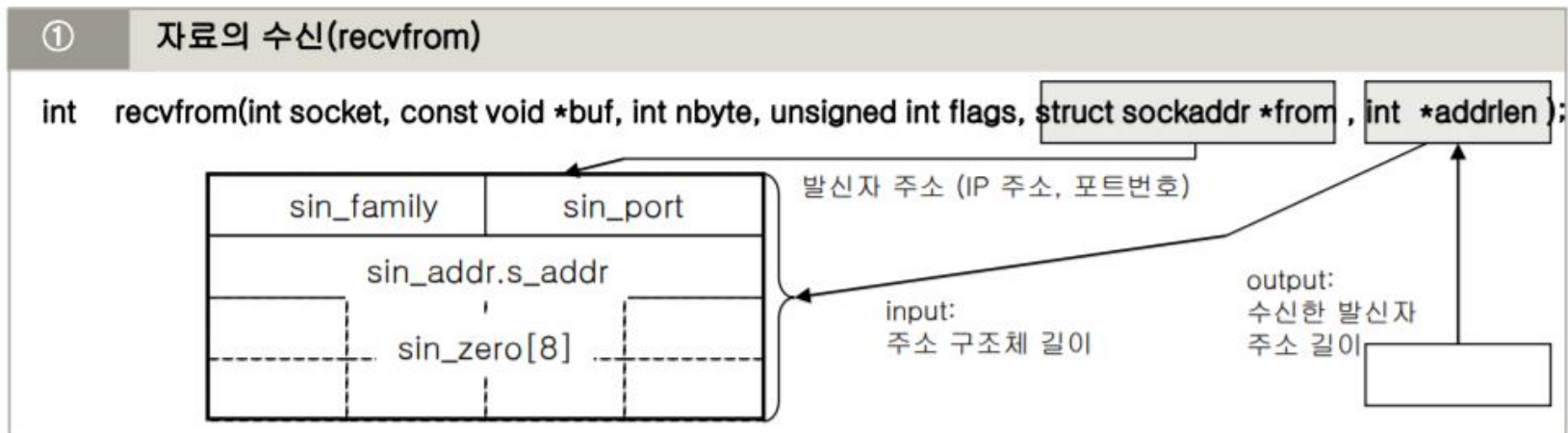
- ① 오류시 재전송을 위한 기능을 제공하지 않음 (손실 없이 전송된 것을 보장하지 않음)
- ② TCP 처럼 송신버퍼 여유공간 부족에 따른 차단(Blocking) 현상 없음
- ③ 자료의 크기가 0인 자료 전송 가능 (recvfrom 함수 호출시 반환 값 0)

```
n = sendto(s, buf, 0, 0, (struct sockaddr *)&s_addr, sizeof(s_addr));
```


소켓 프로그래밍

UDP 서버 프로그램의 기능 수행

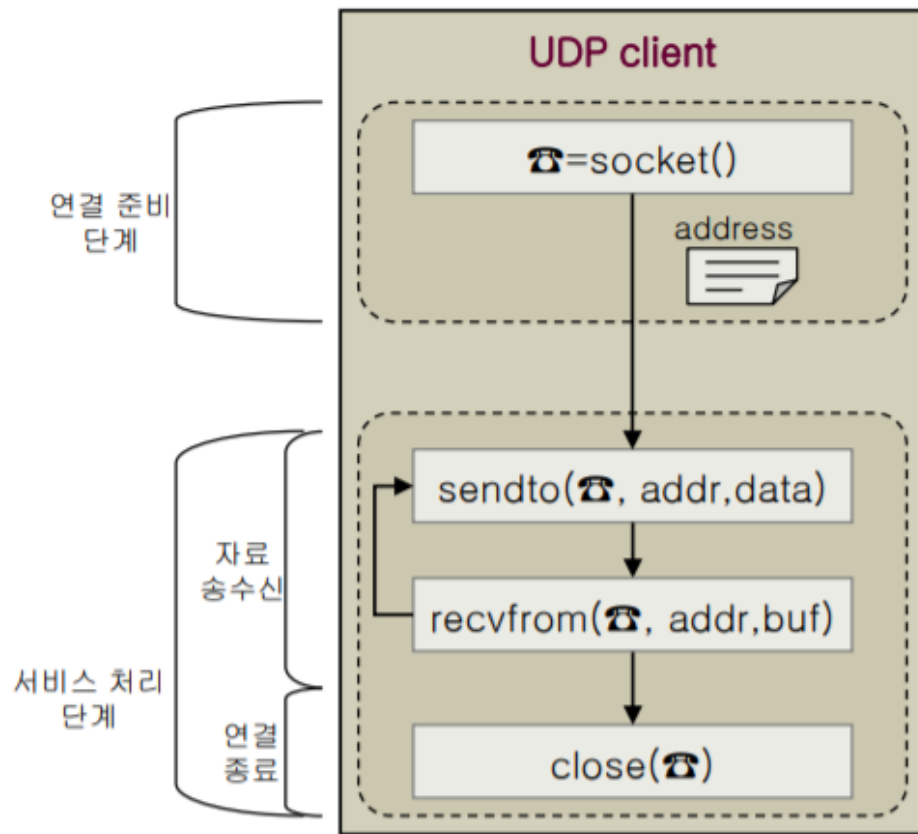
[서비스 처리 단계]



- ① 여러 다른 클라이언트로 부터 수신한 자료의 발신자를 구분할 수 있도록 메시지 경계 유지
- ② recvfrom() 호출시 수신 자료 중 경계가 구분된 첫 번째 자료 nbyte를 가져옴
- ③ 경계가 구분된 첫 번째 자료의 크기가 nbyte보다 클 경우는 나머지 바이트 자료를 버림
- ④ addrlen : 입력 인자로 자료 발신자 주소 저장을 위한 struct sockaddr 의 길이로 사용
출력 인자로 수신한 자료 발신자의 주소 길이 값 저장용으로 사용
% addrlen의 자료형이 정수형의 포인터(int *)임에 주의
- ⑤ TCP의 경우 read() 함수의 반환 값이 0 이면 상대방이 종료 했다는 의미,
UDP에서 recvfrom() 함수의 반환 값이 0 이면, 자료 크기가 0인 자료 수신 의미
- ⑥ 자료 발신지를 알 필요가 없을 경우 5, 6 번째 인자를 NULL 사용
n = recvfrom(s, buf, sizeof(buf), 0, NULL, NULL);

소켓 프로그래밍

UDP 클라이언트 프로그램의 기능 수행



[연결 준비 단계]

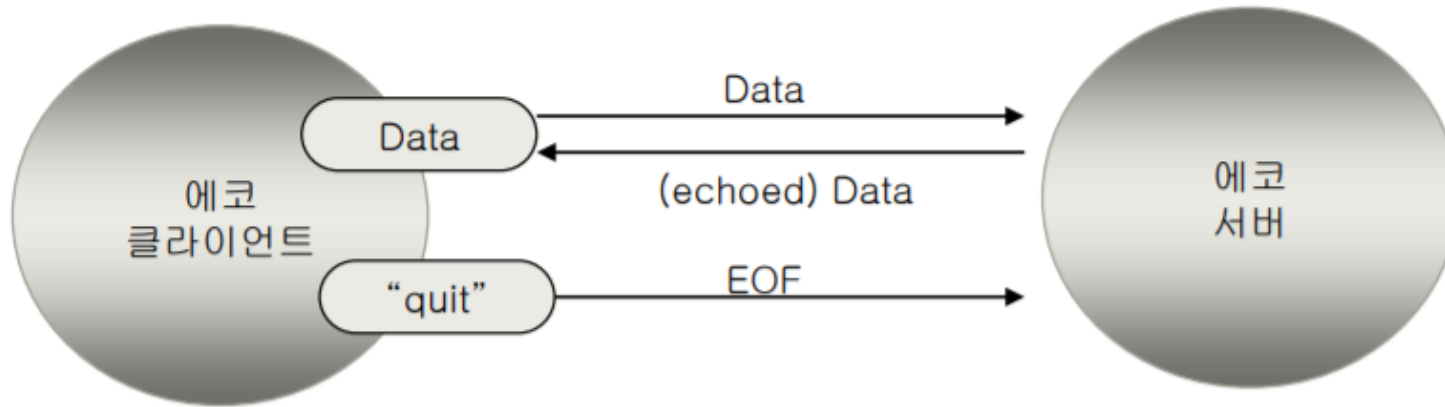
- ① 소켓 생성(socket) : 네트워크 통신을 위해 소켓을 생성한다.

[서비스 처리 단계]

- ① 자료의 송수신(recvfrom/sendto) : UDP 클라이언트와 자료 송수신
- ② 연결 종료(close/shutdown) : 클라이언트와의 연결 소켓 종료

소켓 프로그래밍

UDP 기반 에코(Echo) 프로그램의 구현



- ① 클라이언트는 사용자로부터 데이터를 받아 서버로 보낸다.
- ② 서버는 클라이언트로부터 받은 동일한 데이터를 클라이언트로 다시 보낸다.
- ③ 사용자가 종료를 의미하는 문자열 quit를 입력하면 클라이언트는 서버와의 연결을 종료한다.

소켓 프로그래밍

UDP 에코 서버 소켓 프로그램

```
1 //echo_server_UDP.c
2 #include <stdio.h>
3 #include <sys/types.h>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6
7 main(int argc, char *argv[ ])
8 {
9     int sd;
10    struct sockaddr_in s_addr, c_addr;
11    char buff[BUFSIZ];
12    int n, n_recv;
13    int addr_len;
14
15    sd = socket(AF_INET, SOCK_DGRAM, 0);
16
17    bzero(&s_addr, sizeof(s_addr));
18    s_addr.sin_family = AF_INET;
19    s_addr.sin_addr.s_addr = htonl(INADDR_ANY);
20    s_addr.sin_port = htons(9200);
21
```

```
22     if(bind(sd, (struct sockaddr *) &s_addr, sizeof(s_addr)) < 0){
23         fprintf(stderr, "bind( ) error");
24         exit(-2);
25     }
26
27     while(1) {
28         fprintf(stderr, "waitingWn");
29
30         ① addr_len = sizeof(c_addr);
31         if((n_recv = recvfrom(sd, buff, sizeof(buff), 0,
32                               (struct sockaddr *)&c_addr, &addr_len)) < 0) {
33             fprintf(stderr, "recvfrom( ) error");
34             exit(-3);
35         }
36
37         ② if((n = sendto(sd, buff, n_recv, 0,
38                       (struct sockaddr *) &c_addr, sizeof(c_addr))) < 0){
39             fprintf(stderr, "sendto( ) error");
40             exit(-3);
41         }
42         ③ close(sd);
43     }
```

① 클라이언트로부터 자료를 수신

② 클라이언트로부터 수신한 동일 데이터를 클라이언트로 다시 보냄

③ 서버 종료

소켓 프로그래밍

UDP 에코 클라이언트 소켓 프로그램

```
1 //echo_client_UDP.c
2 #include <stdio.h>
3 #include <sys/types.h>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6
7 main(int argc, char *argv[ ])
8 {
9     int sd;
10    struct sockaddr_in s_addr;
11    char sndBuffer[BUFSIZ];
12    int n, n_send;
13    int addr_len;
14
15    sd = socket(AF_INET, SOCK_DGRAM, 0);
16
17    bzero(&s_addr, sizeof(s_addr));
18    s_addr.sin_family = AF_INET;
19    s_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
20    s_addr.sin_port = htons(9200);
21
22    while(1) {
23
24        fprintf(stderr, "waitingWn");
25
```

```
26        if((n = read(0, sndBuffer, BUFSIZ)) > 0) {
27
28            sndBuffer[n] = '\0';
29            if(!strcmp(sndBuffer, "quitWn")) break;
30
31            printf("original Data : %s", sndBuffer);
32
33            if ((n_send = sendto(sd, sndBuffer, strlen(sndBuffer), 0,
34                               (struct ockaddr *) &s_addr, sizeof(s_addr))) < 0){ ①
35                fprintf(stderr, "sendto( ) error");
36                exit(-3);
37            }
38
39            addr_len = sizeof(s_addr);
40            if((n = recvfrom(sd, sndBuffer, sizeof(sndBuffer), 0,
41                           NULL, NULL)) < 0) {
42                fprintf(stderr, "recvfrom( ) error");
43                exit(-3);
44            }
45            sndBuffer[n] = '\0';
46
47            printf("echoed Data : %s", sndBuffer);
48
49            }
50        }
51    }
52    close(sd); ③
53 }
```

① 서버로 자료를 송신

② 서버가 보내온(echoed) 데이터를 수신

③ 클라이언트 소켓 종료

소켓 프로그래밍

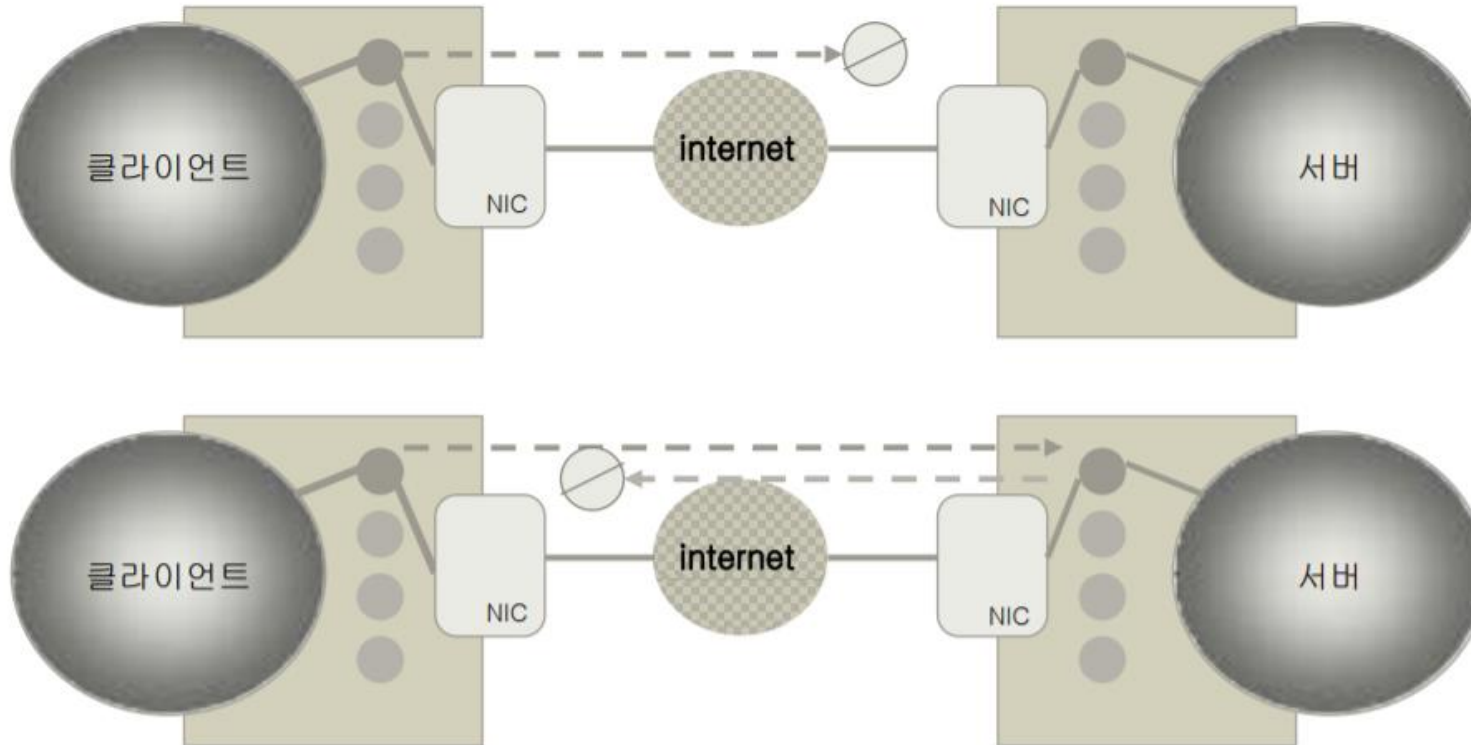
UDP 에코 클라이언트 서버 소켓 프로그램 실행

실행	
실행 터미널-1 / 클라이언트측 실행	실행 터미널-2 / 서버측 실행
<pre>\$ gcc -o echo_client_UDP echo_client_UDP.c \$ echo_client_UDP waiting UDP echo test original Data : UDP echo test echoed Data : UDP echo test waiting quit \$</pre>	<pre>\$ gcc -o echo_server_UDP echo_server_UDP.c \$ echo_server_UDP waiting ■</pre>

소켓 프로그래밍

UDP 에서의 신뢰성 문제

자료 전송 중의 손실



- ① recvfrom() 함수로 오지 않는 응답(자료)를 무한히 기다리는 현상 발생
- ② 대기 시간을 설정하여 무한 대기를 방지

소켓 프로그래밍

UDP 에서의 신뢰성 문제

자료 전송 중의 손실

UDP 에코 클라이언트 소켓 프로그램

```
1 //echo_client_UDPalarm.c

~~ 생략 ~~

19 int status;
20 struct sigaction act;
21
22 act.sa_handler = sigAlarm;
23 sigemptyset(&act.sa_mask);
24 act.sa_flags = 0;
25
26 status = sigaction(SIGALRM, &act, 0);
27
28 sd = socket(AF_INET, SOCK_DGRAM, 0);
29
~~ 생략 ~~

45
46 if((n_send = sendto(sd, sndBuffer, strlen(sndBuffer), 0,
47     (struct sockaddr *) &s_addr, sizeof(s_addr))) < 0){
48     fprintf(stderr, "sendto( ) error");
49     exit(-3);
50 }
```

① 시그널 설정

② 2초 후에 시그널 발생 설정

```
51 alarm(2);
52
53 addr_len = sizeof(s_addr);
54 fprintf(stderr, "socket recvfrom enterWn");
55 if((n = recvfrom(sd, sndBuffer, sizeof(sndBuffer), 0,
56     NULL, NULL)) < 0) {
57     if(errno == EINTR)
58         fprintf(stderr, "socket timeoutWn");
59     else
60         fprintf(stderr, "recvfrom errorWn");
61 } else {
62     alarm(0);
63     sndBuffer[n] = 'W0';
64     printf("echoed Data : %s", sndBuffer);
65 }
66 }
67 close(sd);
68 }
69
70 void
71 sigAlarm(int signo)
72 {
73     fprintf(stderr, "alarmWn");
74     return;
75 }
```

③ recvfrom()에서 패어난 후 타임아웃이면(EINTR) 종료,
자료 수신이면 alarm 해지 후 자료 출력

④ 시그널 처리기 sigAlarm 정의

소켓 프로그래밍

UDP 에서의 신뢰성 문제

자료 전송 중의 손실

UDP 에코 서버 소켓 프로그램 수정 (응답부분 주석처리)

```
37          //if((n = sendto(sd, buff, n_recv, 0, (struct sockaddr *) &c_addr, sizeof(c_addr))) < 0){
38                      //fprintf(stderr, "sendto( ) error");
39                      //exit(-3);
40          //}
```

실행

실행 터미널-1 / 클라이언트측 실행

```
$ echo_client_UDPalarm
waiting
test
original Data : test
socket recvfrom enter
waiting
alarm
socket timeout
waiting
```

실행 터미널-2 / 서버측 실행

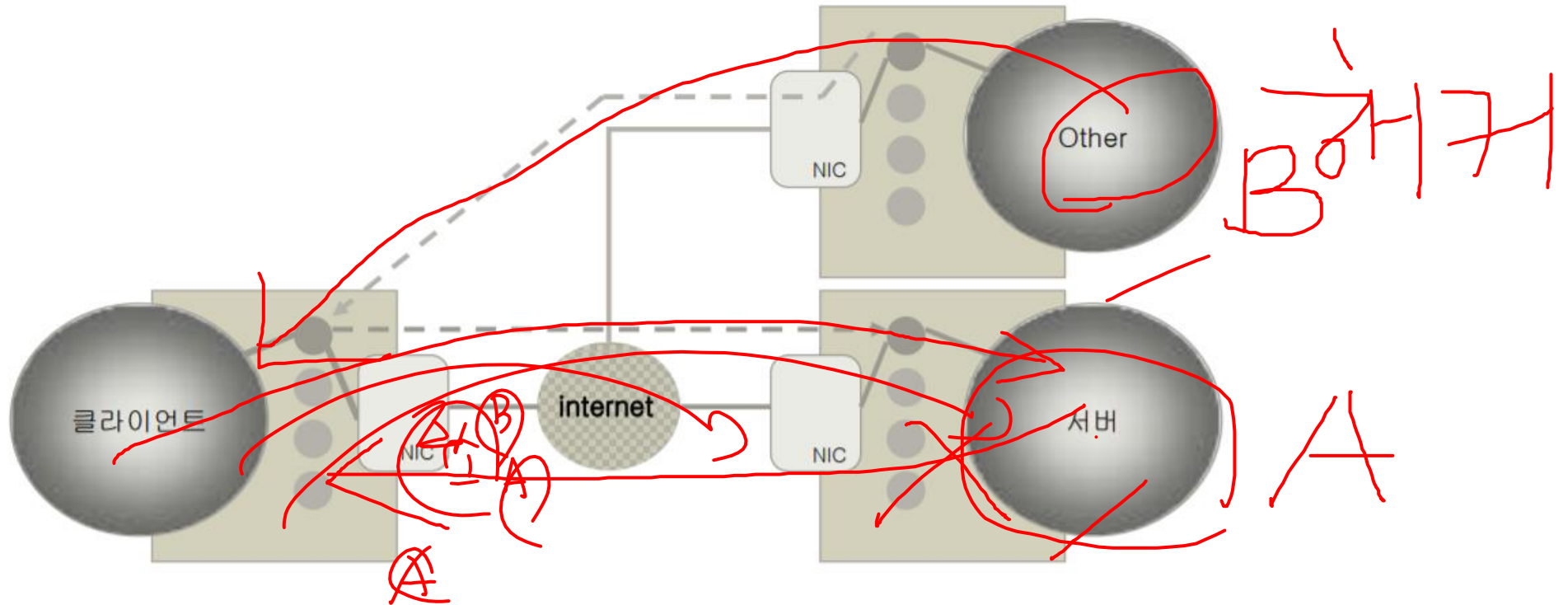
```
$ echo_server_UDPalarm_test
waiting
```



소켓 프로그래밍

UDP 에서의 신뢰성 문제

응답 대상에 대한 확인



① 다른 서버로부터 응답을 받는 경우 발생

② 자료 수신시 자신이 서비스 요청한 서버로부터 응답인지 여부 확인

소켓 프로그래밍

UDP 에서의 신뢰성 문제

UDP 에코 클라이언트 소켓 프로그램

1 //echo_client_UDPconfirm.c

~~ 생략 ~~

```
16 sd = socket(AF_INET, SOCK_DGRAM, 0);
17
18 bzero(&s_addr, sizeof(s_addr));
19 s_addr.sin_family = AF_INET;
20 s_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
21 s_addr.sin_port = htons(9200);
22
23 while(1) {
24
25     fprintf(stderr, "waiting\n");
26
27     if((n = read(0, sndBuffer, BUFSIZ)) > 0) {
28
29         sndBuffer[n] = 'W0';
30         if(!strcmp(sndBuffer, "quit\n")) break;
31
32         printf("original Data : %s", sndBuffer);
33     }
```

```
34 if((n_send = sendto(sd, sndBuffer, strlen(sndBuffer), 0,
35                     (struct sockaddr *)&s_addr, sizeof(s_addr))) < 0){
36     fprintf(stderr, "sendto( ) error");
37     exit(-3);
38 }
39
40 addr_len = sizeof(c_addr);
41 if((n = recvfrom(sd, sndBuffer, sizeof(sndBuffer), 0,
42                 (struct sockaddr *)&c_addr, &addr_len)) < 0) {
43     fprintf(stderr, "recvfrom( ) error");
44     exit(-3);
45 }
46
47 if(memcmp(&c_addr, &s_addr, addr_len) != 0) {
48     fprintf(stderr, "reply from %s/%d (ignored)\n",
49             inet_ntoa(c_addr.sin_addr.s_addr), ntohs(c_addr.sin_port));
50     continue;
51 }
52
53 sndBuffer[n] = 'W0';
54
55 printf("echoed Data : %s", sndBuffer);
56 }
```

도착

응답 대상에 대한 확인

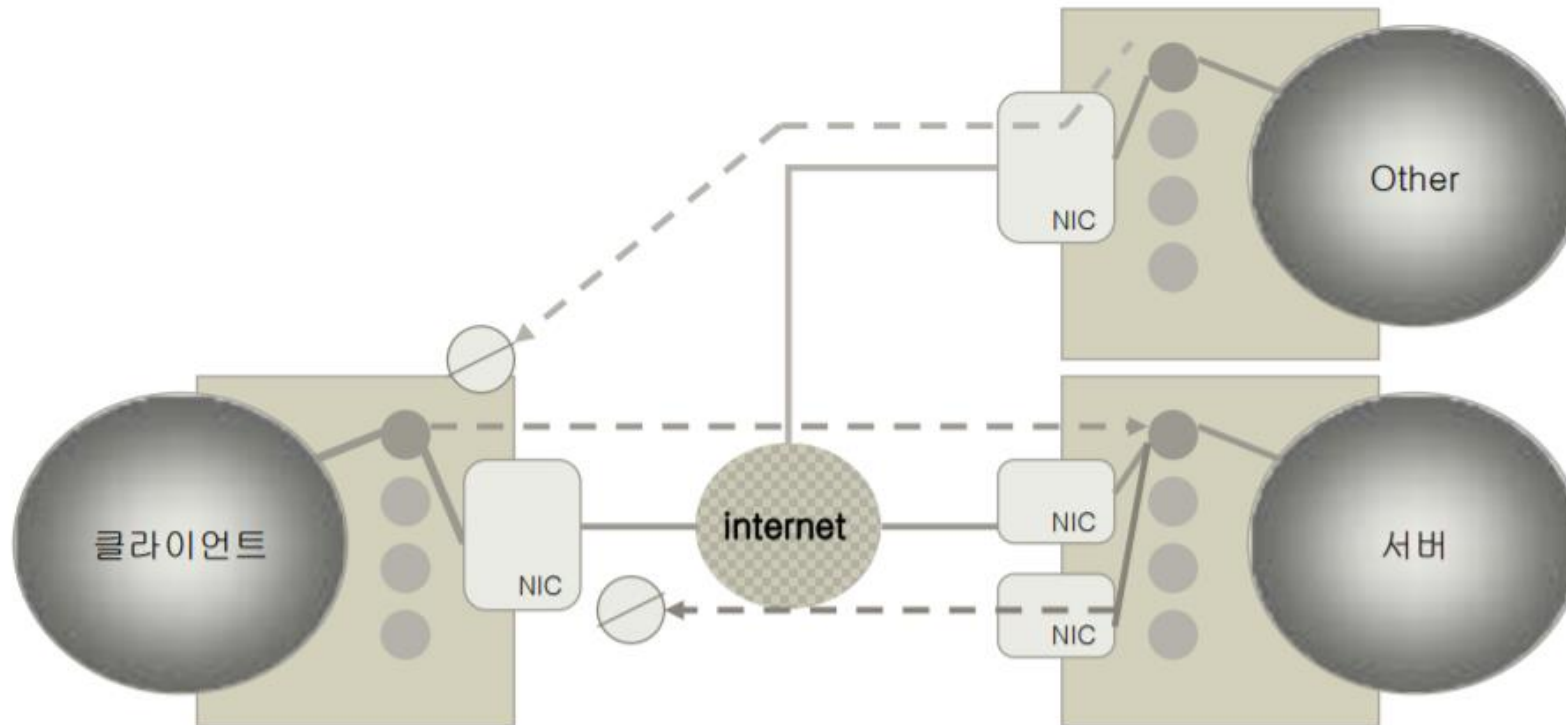
반응

① 자료 수신시 서비스 요청한 서버로부터의 송신 여부 확인

소켓 프로그래밍

UDP 에서의 신뢰성 문제

응답 대상에 대한 확인

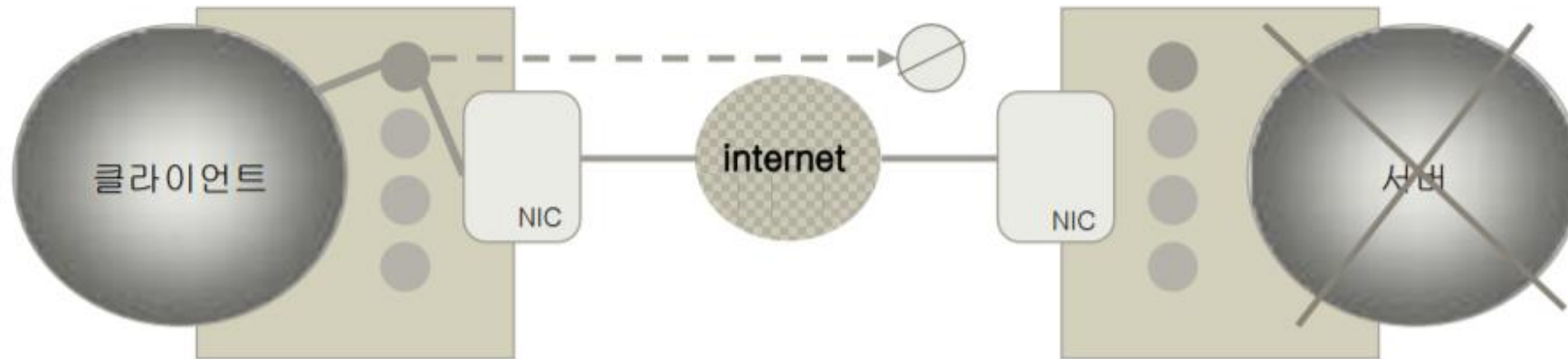


- ① Nic가 여러 개인 경우 요청한 IP와 다른 주소의 IP에서 응답하는 경우 발생
- ② DNS 서비스를 이용하여 도메인 이름의 동일성 확인

소켓 프로그래밍

UDP 에서의 신뢰성 문제

서버의 미실행 오류

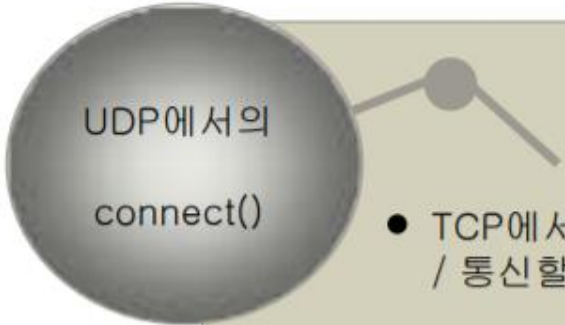


- ① 서버가 실행 중이 아닌 상태에서 클라이언트가 자료를 전송하여도 성공여부 확인 안됨
- ② 클라이언트는 오지 않을 서버로부터의 응답을 무한히 기다림
- ③ Connect 함수를 이용하여 오류 처리

소켓 프로그래밍

UDP 에서의 신뢰성 문제

서버의 미실행 오류



UDP에서의
connect()

- TCP에서 처럼 연결 설정과정(3 방향 핸드셰이킹)이 없다.
/ 통신할 상대방 주소를 OS에 단지 알려 줌
- Connect 후 자료 송수신시 상대방을 지정할 필요가 없다.
/ read(), write()나 sendto() 시 목적지 주소가 NULL인 함수를 사용할 수 있다.
- 비동기 오류(Asynchronous Error) 발생
/ 클라이언트가 서버로 자료를 보내면(sendto()/write()) 수신측에서 서버가 미실행으로 수신할 수 없음을 알리는 ICMP "port unreachable" 반환
/ 다음 소켓 시스템 호출시 오류 반환
/ 일부 OS의 경우는 기능 제공을 하지 않을 수 있음

소켓 프로그래밍

UDP 에서의 신뢰성 문제

서버의 미실행 오류

UDP 에코 클라이언트 소켓 프로그램

```
1 //echo_client_UDPconnect.c
2 #include <stdio.h>
3 #include <sys/types.h>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6
7 main(int argc, char *argv[ ])
8 {
9     int sd;
10    struct sockaddr_in s_addr;
11    char sndBuffer[BUFSIZ];
12    int n, n_send;
13    int addr_len;
14
15    sd = socket(AF_INET, SOCK_DGRAM, 0);
16
17    bzero(&s_addr, sizeof(s_addr));
18    s_addr.sin_family = AF_INET;
19    s_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
20    s_addr.sin_port = htons(9000);
21
22    ① connect(sd, (struct sockaddr *) &s_addr, sizeof(s_addr));
23
```

① connect 사용

② write시 서버가 실행 중이 아님을 알아냄 (ICMP "port unreachable") - 비동시 오류 발생(Asynchronous Error)

③ write 다음 소켓 호출 함수인 read시 오류 감지

```
24 while(1) {
25
26     fprintf(stderr, "waitingWn");
27
28     if((n = read(0, sndBuffer, BUFSIZ)) > 0 ) {
29
30         sndBuffer[n] = 'W0';
31         if(!strcmp(sndBuffer, "quitWn")) break;
32
33         printf("original Data : %s", sndBuffer);
34
35         if((n_send = write(sd, sndBuffer, strlen(sndBuffer))) < 0){ ②
36             fprintf(stderr, "write error");
37             exit(-3);
38         }
39
40         addr_len = sizeof(s_addr);
41         if((n = read(sd, sndBuffer, sizeof(sndBuffer))) < 0) { ③
42             fprintf(stderr, "read error");
43             exit(-3);
44         }
45         sndBuffer[n] = 'W0';
46
47         printf("echoed Data : %s", sndBuffer);
48     }
49 }
50 close(sd);
51 }
```


소켓 프로그래밍

UDP 에서의 신뢰성 문제

서버의 미실행 오류

connect를 이용한 UDP 에코 클라이언트 소켓 프로그램

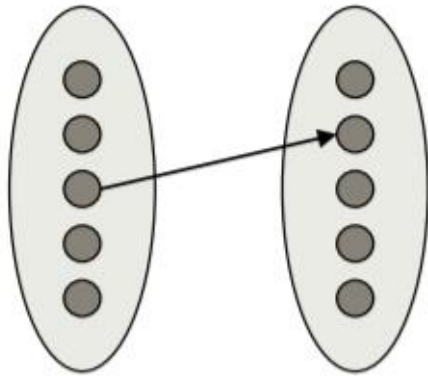
실행

실행 터미널-1 / 클라이언트측 실행

```
$ connect
waiting
UDP connect error
original Data : UDP connect error
read error$
```

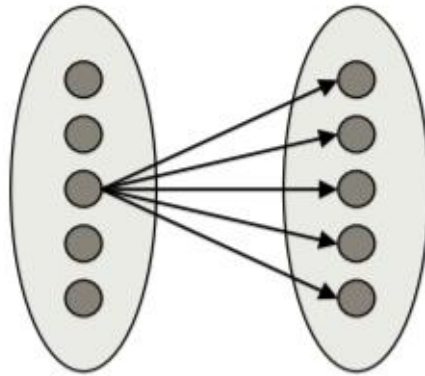
소켓 프로그래밍

UDP를 이용한 브로드캐스팅(Broadcasting)



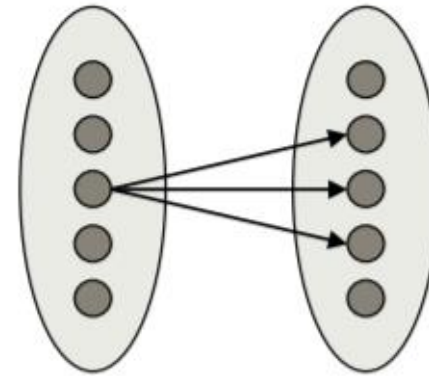
유니캐스트(unicast)

TCP, UDP



브로드캐스트(broadcast)

UDP only

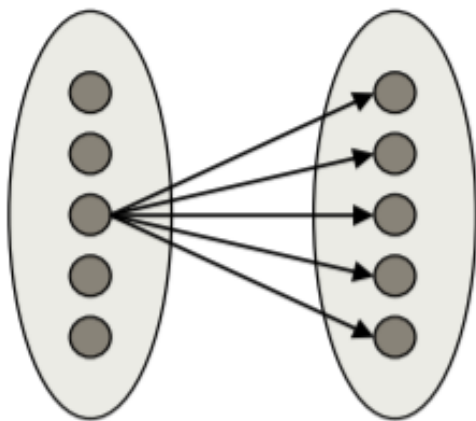


멀티캐스트(multicast)

UDP only

소켓 프로그래밍

UDP를 이용한 브로드캐스팅(Broadcasting)



브로드캐스트(broadcast)

UDP only

① 소켓에 대한 브로드캐스팅 활성화

```
int on = 1; // true
setsockopt( sd, SOL_SOCKET, SO_BROADCAST, &on, sizeof(on) );
```

② 브로드캐스팅되는 UDP 데이터를 수신하려는 클라이언트 프로그램은 해당 브로드캐스팅 주소와 포트에 대해 우선 연결되어(bind) 있어야 한다.

LAN의 서브 네트워크 주소가 192.168.42인 경우
브로드캐스팅 주소는 192.168.42.255이다.

소켓 프로그래밍

UDP를 이용한 브로드캐스팅(Broadcasting)

자신이 속한 네트워크 인터페이스 브로드캐스팅 주소 확인

실행

실행 터미널-1 / 클라이언트측 실행

```
$ ./ifconfig
eth0  Link encap:Ethernet  HWaddr 00:1E:4F:17:D7:F4
      inet addr:203.249.39.141  Bcast:203.249.39.159 Mask:255.255.255.224
      inet6 addr: fe80::21e:4fff:fe17:d7f4/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:240083044 errors:0 dropped:0 overruns:0 frame:0
      TX packets:430634044 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:3853940293 (3.5 GiB)  TX bytes:327351381 (312.1 MiB)
      Interrupt:169 Memory:f8000000-f8012100
lo    Link encap:Local Loopback
      inet addr:127.0.0.1  Mask:255.0.0.0
      inet6 addr: ::1/128 Scope:Host
      UP LOOPBACK RUNNING  MTU:16436  Metric:1
      RX packets:439642 errors:0 dropped:0 overruns:0 frame:0
      TX packets:439642 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:0
      RX bytes:147334731 (140.5 MiB)  TX bytes:147334731 (140.5 MiB)
```

소켓 프로그래밍

UDP를 이용한 브로드캐스팅(Broadcasting)

브로드캐스터 제작

UDP 브로드캐스터 소켓 프로그램

```
1 //UDPbcaster.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <sys/types.h>
6 #include <sys/socket.h>
7 #include <netinet/in.h>
8 #include <signal.h>
9 #include <errno.h>
10
11 main(int argc, char* argv[])
12 {
13     int sd;
14     struct sockaddr_in s_addr;
15     char sndBuffer[BUFSIZ];
16     int n, n_send, status;
17
18     int on = 1;
19
20     if(argc != 3) {
21         fprintf(stderr,
22             "usage: UDPbacster broadcast_ip_addr portWn");
23         exit(-1);
24     }
25
26     sd = socket(AF_INET, SOCK_DGRAM, 0);
```

```
27     bzero(&s_addr, sizeof(s_addr));
28     s_addr.sin_family = AF_INET;
29     s_addr.sin_addr.s_addr = inet_addr(argv[1]);
30     s_addr.sin_port = htons(atoi(argv[2]));
31
32     if ((status = setsockopt(sd, SOL_SOCKET, SO_BROADCAST, &on,
33                             sizeof(on))) != 0) {
34         printf("setsockopt errorWn");
35         exit(-1);
36     }
37
38     while(1) {
39         fprintf(stderr, "Type broadcasting data : ");
40
41         if ( (n = read(0, sndBuffer, BUFSIZ)) > 0 ) {
42             sndBuffer[n-1] = 'W0';
43
44             if ((n_send = sendto(sd, sndBuffer, strlen(sndBuffer), 0,
45                                 (struct sockaddr *)&s_addr, sizeof(s_addr))) < 0){
46                 fprintf(stderr, "sendto() error");
47                 exit(-3);
48             }
49
50             if (!strncmp(sndBuffer, "quit", 4)) break;
51         }
52     }
53     close(sd);
54 }
```

① 소켓 옵션 SO_BROADCAST를 설정

② 자료를 입력받아 브로드캐스트

소켓 프로그래밍

UDP를 이용한 브로드캐스팅(Broadcasting)

브로드캐스터 제작

UDP 리더 소켓 프로그램

```
1 //echo_client_UDPbcast.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <sys/types.h>
6 #include <sys/socket.h>
7 #include <netinet/in.h>
8 #include <errno.h>
9
10 main(int argc, char* argv[])
11 {
12     int sd;
13     struct sockaddr_in s_addr, c_addr;
14     char rcvBuffer[BUFSIZ];
15     int n, n_send;
16     int addr_len;
17
18     int status;
19     int on = 1;
20
21     if(argc != 2) {
22         printf("usage: UDPReader portWn");
23         exit(-1);
24     }
25
26     sd = socket(AF_INET, SOCK_DGRAM, 0);
27
```

```
28     bzero(&s_addr, sizeof(s_addr));
29     s_addr.sin_family = AF_INET;
30     s_addr.sin_addr.s_addr = INADDR_ANY;
31     s_addr.sin_port = htons(atoi(argv[1]));
32
33     if (bind(sd, (struct sockaddr *)&s_addr, sizeof(s_addr)) != 0) {
34         fprintf(stderr, "bind errorWn");
35         exit(-2);
36     }
37
38     while(1) {
39         addr_len = sizeof(s_addr);
40         if ((n = recvfrom(sd, rcvBuffer, sizeof(rcvBuffer), 0,
41             (struct sockaddr *)&c_addr, &addr_len)) < 0) {
42             fprintf(stderr, "recvfrom error");
43             exit(-3);
44         }
45         rcvBuffer[n] = '\0';
46
47         if(!strncmp(rcvBuffer, "quit", 4))
48             break;
49
50         printf("Broadcasted from %s [%s]Wn", inet_ntoa(c_addr.sin_addr.s_addr), rcvBuffer);
51     }
52     close(sd);
53 }
```

① 브로드캐스팅된 자료 수신

소켓 프로그래밍

UDP를 이용한 브로드캐스팅(Broadcasting)

브로드캐스터 제작

실행

실행 터미널-1 / UDPbcaster 실행

```
$ gcc -o UDPbcaster UDPbcaster.c
$ UDPbcaster 203.249.39.159 9888
Type broadcasting data : broadcast test
Type broadcasting data : send 12345
Type broadcasting data : quit
$
```

실행 터미널-2 / UDPreader 실행

```
$ gcc -o UDPreader UDPreader.c
$ UDPreader 9888
Broadcasted from 203.249.39.141 [broadcast test]
Broadcasted from 203.249.39.141 [send12345]
$
```

참고

UDPbcaster 실행시 인자로 주는 브로드캐스팅 주소는 독자 여러분의 호스트가 연결된 네트워크에서 따로 확인하여 사용해야 한다.

```
$ bcaster 203.249.39.159 9888
```

브로드캐스트 주소는 ifconfig 명령으로 알아볼 수 있다.

소켓 프로그래밍

9주 과제

1. Echo_server를 제작할때 TCP를 이용할때와 UDP를 이용할때의 차이점을 설명하시오.
(read/write 함수와 sendto/recvfrom 함수의 차이점, accept, ...)
2. UDP를 이용하여 echo_server 를 구동하면 어떤 문제점이 발생할수 있는지 그리고 그 문제점을 어떻게 해결해야 하는지 설명하시오. (문제점이 발생하는 상황하는 실습예제를 실행하고 화면 캡처한다)
3. echo_client_UDPconfirm.c 를 변경하여 아래와 같이 도착IP주소를 출력한다.

```
user@raspberrypi:~/ch6/9week/yb $ ./client_confirm2 9000
waiting
1234
original Data : 1234
echoed Data(127.0.0.1) : 1234
waiting
```